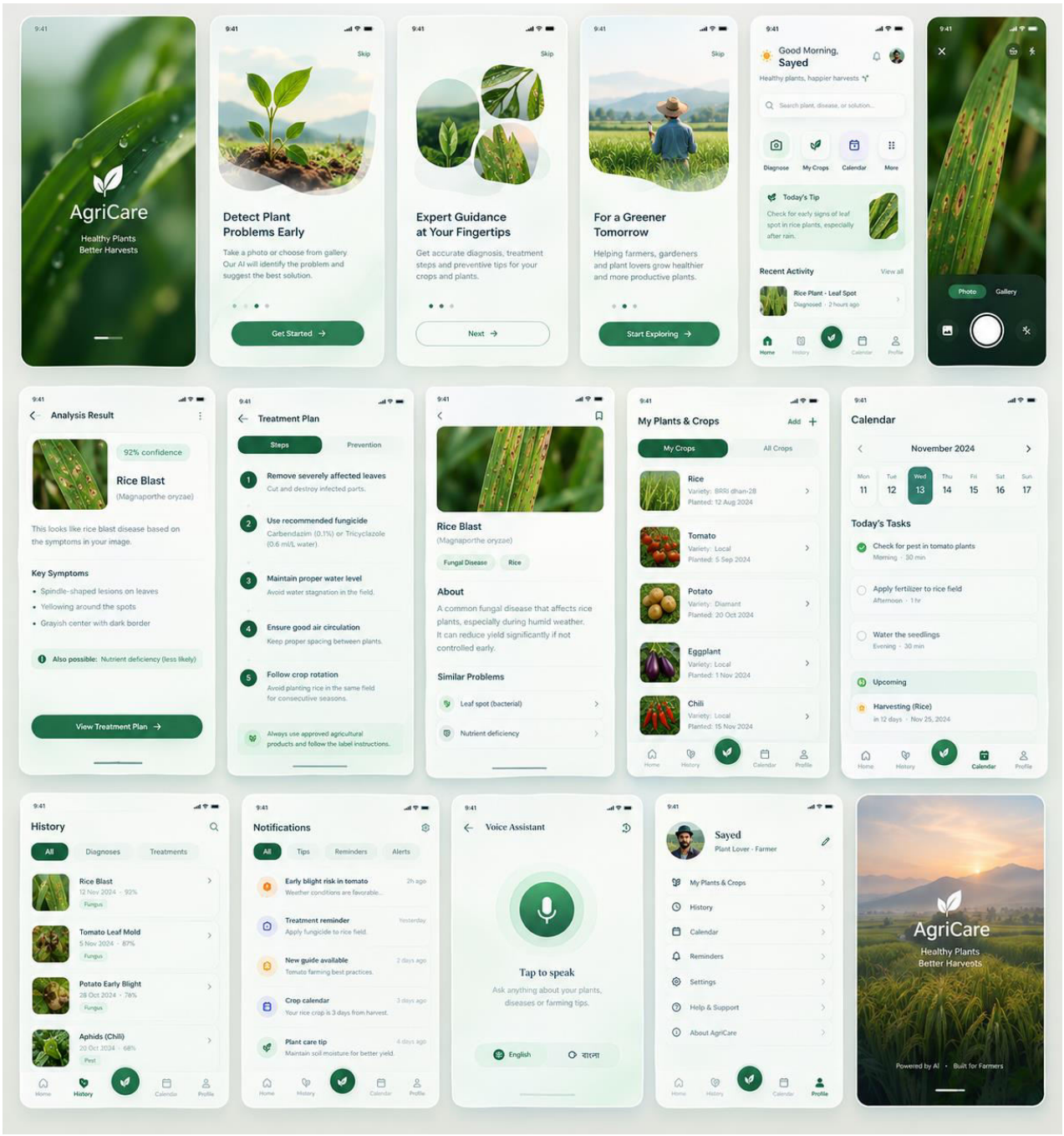
AgriCare

MASTER PRODUCT REQUIREMENTS DOCUMENT

Zero-budget-first Android plant health assistant
English-first interface • Bangla localization-ready • Google AI Studio implementation



MASTER VISUAL REFERENCE — Use this latest AgriCare mockup as the primary visual source of truth. Recreate its visual language and screen hierarchy; do not treat the collage as permission to fabricate features or use fake data.

0. Instructions to Google AI Studio — Read Before Building

Build a real, functional Android-first application called AgriCare. This document is the source of truth for product scope, behavior, architecture, design, navigation, data persistence, localization, and acceptance criteria. The attached latest mockup image is the master UI reference. Inspect it and reproduce its overall visual language: airy layouts, restrained sage/forest greens, soft neutral surfaces, subtle translucent layers, realistic crop photography, rounded but disciplined cards, calm typography, and polished spacing.

- Do not generate a generic AI dashboard, a flower shop, a gardening marketplace, a luxury-fashion interface, or a decorative landing page. This is a practical crop-health tool.
- Do not stop after creating a visual prototype. Every visible control must perform a real action, navigate to a real screen, or be clearly disabled with a reason. No dead buttons.
- Do not use fake/mock diagnosis results, fabricated weather, fake confidence scores, fake notifications, or invented treatment advice in production flows. Use clearly labelled demo data only in an optional demo mode.
- Do not replace the whole app or redesign it arbitrarily after implementing a screen. Follow this PRD and the master mockup.
- Do not use a paid AI API. Use Gemini API via an available free tier for development/testing, with quotas and availability treated as variable. The app must degrade gracefully when the quota is exhausted.
- Never embed a production Gemini API key in a public client build. For a zero-budget prototype, use a local development configuration only and explain the exposure risk. For public release, use a minimal server-side proxy only if a genuinely free hosting option and its limits are acceptable; otherwise do not ship a publicly extractable key.
- Use the latest stable, compatible Android/web stack that AI Studio supports. If generating a web-based prototype first, keep the architecture responsive and make Android packaging/PWA constraints explicit rather than claiming native capabilities that are not implemented.
- Implement in small phases. Before each phase, inspect existing code and preserve completed working functionality. Do not rewrite working screens unnecessarily.

1. Product Overview

AgriCare is an English-first, Bangla-localizable plant and crop health assistant for farmers, gardeners, and small growers, with an initial focus on common Bangladesh crops. A user takes or selects a plant photo; Gemini vision analyzes visible symptoms and returns a cautious, structured assessment, possible causes, next steps, prevention guidance, and clear advice about when to consult a local agricultural officer. The app also stores crops and diagnosis history locally, offers a curated crop-health library, and supports reminders.

1.1 Product promise

“Understand plant problems earlier. Get practical next steps.” AgriCare must be useful, trustworthy, calm, and understandable—not pretend to replace a qualified agronomist or laboratory diagnosis.

1.2 Target users

- Primary: Bangladeshi smallholder farmers who need understandable, practical crop-health guidance.
- Secondary: home growers, gardeners, students, and agricultural learners.
- Design for low-to-mid-range Android phones, variable network quality, one-handed use, readable text, and users who may prefer Bangla after localization is enabled.

1.3 Zero-budget principle

Build and test with no paid services. Prefer local persistence and bundled educational content. Gemini API free-tier access is subject to account eligibility, region, model availability, rate limits, quotas, and changing terms. Never promise unlimited free image analysis. Do not require sign-in for the MVP.

2. MVP Scope and Non-Goals

2.1 MVP must include

- Splash screen and first-run onboarding.
- English default language with complete localization architecture and a working English/Bangla language switch.
- Home dashboard with clear primary action to diagnose a plant.
- Camera capture and gallery image selection.
- Image validation, preview, retake/crop if supported, and explicit consent before uploading a photo for AI analysis.
- Gemini-powered structured analysis with loading, retry, timeout, offline, quota-exceeded, and uncertain-result states.
- Analysis result, symptoms, alternative causes, practical next steps, prevention, safety notes, and follow-up questions.
- Treatment/guide details and crop library with locally bundled starter content.
- My Plants & Crops: add/edit/archive crop records and view details.
- Diagnosis and treatment history saved on device.
- Calendar/reminders for user-created crop tasks, with local notifications only where platform support and permissions exist.
- Profile/settings, language selection, privacy information, app information, and clear-data controls.
- Persistent storage, empty states, error states, accessibility basics, and test coverage for core flows.

2.2 Explicitly out of scope for first release

- Paid subscriptions, ads, payment, marketplace, e-commerce, community feed, social login, or required cloud accounts.
- Guaranteed disease detection, laboratory-grade diagnosis, guaranteed yield predictions, or automatic pesticide prescriptions.
- A claim that all features work fully offline. Gemini image analysis requires network access unless a separate on-device model is implemented.
- Live weather, satellite analysis, automatic location tracking, or push-notification infrastructure unless separately approved and implemented with a free, verified source.
- Training a custom large model from scratch as part of MVP.

3. Brand and Visual Direction

Use the attached latest mockup collage as the master visual reference. The design should feel premium and exceptionally smooth, but remain practical and restrained. Visual quality comes from layout, typography, consistency, imagery, motion, and detail—not from adding more cards or gradients.

3.1 Visual tokens

- Primary forest green: approximately #176B4D; deep green for primary actions and brand marks.
- Sage surface: approximately #EAF3EC; warm off-white: approximately #F8FAF7; primary text: #20352B; secondary text: #6C7D73.
- Use a subtle green-tinted gradient only for selected hero areas. Avoid gradients behind every card.
- Card radius: 16–22 dp depending on component; buttons: 14–18 dp. Keep a consistent radius scale rather than random rounding.
- Spacing system based on 4 dp increments: 4, 8, 12, 16, 20, 24, 32. Default screen horizontal padding 20–24 dp depending on device width.
- Typography: use a clean sans-serif family, strong readable hierarchy, no excessive font weights, no decorative serif display headlines.
- Use real, relevant crop and leaf photography for diagnosis and crop content. Avoid random flowers, mountains, landscape banners, or generic stock photos as filler.
- Icons must be consistent in style, stroke, optical size, and alignment. Prefer simple line icons and use labels for key navigation.

3.2 Motion and interaction quality

- Use short, subtle transitions (roughly 160–260 ms where appropriate); respect reduced-motion preferences.
- Use gentle fade/slide transitions for screen changes, restrained press feedback, skeletons for actual loading, and smooth progress states.
- Do not animate every card continuously. No distracting parallax, bouncing buttons, or excessive blur.
- Maintain 60 fps target where feasible on common mid-range Android hardware. Avoid oversized blur layers, huge images, and unnecessary recomposition.
- All touch targets should be at least 48 dp where possible. Provide pressed, focused, disabled, loading, success, and error states.

4. Critical Header, Status Bar, and Safe-Area Rules

THIS SECTION IS A HARD REQUIREMENT. Google AI Studio often places top headers too low, leaves oversized blank space above them, or places them under the status bar. Do not guess header positions. Implement a consistent, measured top app-bar system and verify it on real device-size previews.

4.1 Header placement rules

1. Respect Android status-bar and display cutout insets. The header must begin below the status bar, never behind it, unless a deliberate edge-to-edge layout is implemented with correct inset handling.
2. Use one consistent app-bar component for all standard screens. Its content area should generally be 56 dp high, excluding system insets. Do not add arbitrary top margin to individual screens.
3. For standard screens, place the leading back/menu icon and title on the same horizontal baseline. Keep the title vertically centered within the app bar.
4. Use a consistent horizontal inset of 20–24 dp. Align header left edge with the main content grid unless the reference screen explicitly calls for a full-bleed image.
5. Use one clear title hierarchy: screen title first, optional subtitle below only when useful. Do not stack a huge title, breadcrumb, subtitle, and toolbar all above the fold.
6. On home, the greeting row may be more expressive, but it must still respect system insets and align with the dashboard content grid.
7. For camera and immersive photo-preview screens, hide or simplify the standard app bar only when intentional; camera controls must respect cutouts and bottom gesture insets.
8. Avoid double top padding: do not combine systemBars padding, Scaffold top padding, and a manual status-bar spacer if the framework already accounts for the inset.
9. Do not use fixed absolute Y positions for headers. Use safe-area/inset-aware layout primitives and shared components.
10. Verify at minimum: 360×800, 393×852, and a taller device; test with gesture navigation and three-button navigation if applicable.
11. Capture screenshots of Home, Crop Library, Analysis Result, Treatment, Calendar, History, and Settings. Check that no header is too low, clipped, duplicated, or misaligned.

4.2 Header acceptance criteria

- No header overlaps the status bar, notch, camera cutout, or system icons.
- No unexplained large blank strip appears above the title.
- Back buttons, titles, and action icons share a visually consistent centerline and spacing.
- Scrollable content begins below the app bar and never slides underneath it unless the screen is explicitly designed for that behavior.
- Keyboard opening does not shift the app bar unpredictably.
- All screen headers remain visually aligned when the language changes to Bangla and text becomes longer.

5. Navigation Architecture

Use a predictable bottom navigation with five destinations inspired by the reference: Home, My Crops, a prominent central Diagnose action, History, and Profile. Calendar and the crop-health library are reached from Home shortcuts and relevant contextual links. Avoid placing every feature in the bottom bar.

- Home: dashboard and shortcuts.
- My Crops: list, add, edit, crop detail.
- Diagnose: camera/gallery workflow, analysis, result, treatment.
- History: previous analyses and treatments, filters, details.
- Profile: preferences, language, reminders, privacy, support/about.
- Calendar: accessible from Home and crop details; user tasks and reminders.
- Crop Library: browse crops, diseases, symptoms, prevention, and curated guidance.

Navigation must preserve meaningful state when moving back from detail screens. Android back should close dialogs/sheets first, then return to the prior screen, then exit from Home after a normal confirmation policy. No route may lead to a blank screen or app crash.

6. Full Screen-by-Screen Requirements

6.1 Splash Screen

- Show AgriCare leaf mark, wordmark, and a short line such as “Healthy Plants. Better Harvests.” on a calm deep-green botanical background or a simple premium surface.
- Keep it minimal and quick. Do not force a long animation or show a fake progress percentage.
- On every cold launch, show a brief brand splash if platform conventions allow; then route to onboarding only on first launch and Home on later launches.
- Use a real vector/logo asset or clean custom vector, not a screenshot of the logo.

6.2 Onboarding

- Three concise slides: Detect Plant Problems Early; Practical Guidance; Grow with Confidence.
- Use relevant crop photography with controlled crops and generous whitespace. Avoid large decorative landscapes that push content too far down.
- Show Skip, Next, page indicators, and a clear Get Started action. Buttons must work.
- Ask for language choice only if not already set; English is the default. Do not request camera/notification permission before the user tries the related feature.
- Persist onboarding completion. Do not show onboarding again unless user explicitly chooses Reset onboarding.

6.3 Home Dashboard

- Header: time-aware greeting, short supportive subline, notification/profile actions. Use actual local time for Good morning/afternoon/evening; do not hardcode.
- Primary action: Diagnose a Plant, with Camera and Gallery as explicit options.
- Search field for crops, diseases, symptoms, and guides. Search must return useful local content.
- Today’s Tip: a curated, sourced tip with dismiss/refresh behavior; do not invent a daily tip from fake data.
- My Crops preview: actual saved crops, or an intentional empty state with Add your first crop.
- Recent Activity: real locally saved diagnosis records; empty state when none exist.
- Optional reminder preview only if reminders exist. Do not show fabricated weather or fabricated crop stats.
- Keep Home visually light and avoid a grid of too many same-weight cards. The Diagnose action should be the strongest focal point.

6.4 Diagnose Entry

- Let the user choose Take Photo or Choose from Gallery. Explain that the image may be sent to Gemini for analysis before the first online diagnosis.
- Provide a short photography guide: focus on affected area, natural light, include leaf and whole plant if possible, avoid blur.
- Show permission rationale only when necessary. If permission is denied, explain how to continue via gallery or device settings.

6.5 Camera Capture

- Use the real device camera integration available to the chosen framework. Do not fake a live camera with a static image.
- Show a crop/leaf framing guide, capture button, gallery shortcut, close button, and flash control only if actually supported.
- Handle denied permission, no camera available, rotation, lifecycle interruptions, and capture failure.
- After capture, show preview with Retake and Analyze.

6.6 Gallery Picker and Image Preview

- Use the system photo picker where possible. Never request broad gallery access if a scoped picker is sufficient.
- Support common image types, enforce sensible file-size/dimension limits, and show a clear unsupported-file error.
- Allow preview, retake/choose another, and continue. Compress/resize images before API upload to control data use and avoid memory crashes.
- Do not upload anything before the user presses Analyze and has been informed about network processing.

6.7 Analysis Loading and Failure States

- Show a refined, calm progress state with short steps such as Preparing image, Examining visible symptoms, Preparing guidance. These are UI stages, not claims about hidden model reasoning.
- Support timeout, no internet, quota/rate limit, server/API error, invalid API configuration, unsupported image, and empty/unparseable response.
- Provide Retry, Choose another image, and Open offline guide actions as appropriate.
- Do not simulate success after a failed request. Never fill the result screen with hardcoded demo diagnosis.

6.8 AI Analysis Result

- Show image thumbnail, likely issue title, crop if known, and an uncertainty-aware label such as Likely match / Needs more information.
- Display confidence only if the model returns a calibrated value from a validated system. Do not present a model's self-reported percentage as a scientifically calibrated probability. Prefer qualitative confidence and explain limits.
- Sections: What may be happening; visible signs; possible alternative causes; recommended next steps; prevention; when to contact an agricultural officer.
- Show a clear note: image-based suggestions are not a definitive diagnosis. Similar symptoms may have different causes.
- Actions: View Treatment Plan, Ask a Follow-up Question, Save to History, Add to My Crops, Share Report (with user action and privacy awareness), Analyze another image.
- Keep the output structured, concise, and readable. Never output an automatic pesticide dosage unless grounded in verified, crop-specific, locally applicable label guidance.

6.9 Follow-up Q&A

- Let the user ask additional questions about the current diagnosis in plain language. Keep the current image/result context only for the current session unless user saves it.
- Ask targeted clarifying questions when appropriate: crop, plant age, field/watering conditions, where symptoms appear, spread speed, recent weather or fertilizer use.

- Do not ask too many questions at once. Provide quick reply chips only when they are relevant and editable.
- Warn if the follow-up cannot be answered safely without an in-person inspection.

6.10 Treatment Plan

- Present numbered steps, priority, timing, prevention, monitoring, and follow-up signs.
- Separate immediate low-risk actions from optional agricultural products. Recommend physical/cultural controls first where appropriate.
- Any pesticide/fungicide suggestion must be generic unless a verified, current, locally applicable source confirms product, crop, target disease, label, dose, PPE, pre-harvest interval, and local approval. Do not invent brands, dosages, mixing instructions, or repeat intervals.
- Include safety reminders: follow local product labels, wear required protective equipment, keep children/animals away, and seek local expert guidance when uncertain.
- Allow the user to mark a step complete and save the plan. Persist completion locally.

6.11 Crop and Disease Library

- Search and filter by crop, symptom, disease, and category. Starter crops may include rice, potato, tomato, eggplant, chilli, jute, maize, and other locally relevant crops as content quality allows.
- Each guide includes crop, common symptoms, look-alikes, likely causes, prevention, safe management, source attribution, and last-reviewed date when known.
- Show an empty state and a clear search reset. Never invent a source citation or imply content is officially endorsed if it is not.
- Bundled content should work offline. Distinguish verified editorial content from AI-generated suggestions.

6.12 My Crops

- Add crop: crop name, optional variety, planting date, location/field label (optional), notes, and optional photo.
- List cards show crop name, saved date/details, and user-created task summaries—not fabricated health scores.
- Crop detail includes notes, associated diagnoses, tasks/reminders, edit/archive/delete actions.
- Support duplicate names; do not force location access. All edits persist after restart.

6.13 Calendar and Reminders

- Month/week/list views as supported by the framework, with a simple and readable calendar.
- Create/edit/delete reminders with title, date/time, related crop (optional), repeat settings only if implemented correctly, and notification toggle.
- Request notification permission contextually on Android versions that require it. If permission is denied, still show reminders inside the app.
- Handle timezone changes, past reminders, app restart, duplicate scheduling, and cancelled reminders.
- Do not show fake harvest dates or auto-generated agronomy schedules unless user-created or sourced.

6.14 History

- Store real diagnosis history locally: timestamp, image thumbnail or local reference, crop (if known), result summary, uncertainty, and saved treatment checklist.
- Filter by date, crop, and diagnosis status; open details; delete one record; clear all history only after confirmation.
- History remains available offline. If an image is not retained due to storage policy, show a clear placeholder and retain the text report.
- Never send saved history to an external service without a clear user action.

6.15 Notifications

- Show an in-app notification/reminder center backed by real local events. Empty state must be honest.
- Notification items must open the relevant crop, task, or diagnosis. Mark-as-read and clear actions must work.
- Do not ship a test-notification button as a production feature.

6.16 Voice Assistant — optional MVP enhancement

- If supported without paid services, provide speech-to-text using a real platform capability and clear permission prompts. If unsupported, omit the control rather than displaying a dead microphone.
- Voice questions must be editable before sending. Text input is always available as a fallback.
- Do not claim voice works offline unless it has been tested on supported devices.

6.17 Profile and Settings

- Settings: language, appearance (system/light/dark only if all screens support it), notification preferences, image upload/data-use explanation, privacy policy, terms, about, help, clear diagnosis history, export/share data if implemented, reset onboarding.
- Default theme should be light to match the reference. Do not force dark mode on individual tabs.
- Language switch must update every visible string, dialogs, errors, buttons, navigation, dates, accessibility labels, and empty states immediately.
- Include a working data-clear flow with explicit confirmation and an explanation of what will be deleted.

6.18 About, Privacy, and Terms

- Provide actual readable pages. Do not link to blank routes or placeholder screens.
- Privacy page explains image upload to Gemini, local data storage, no required account, retention behavior, and limits of the prototype.
- Terms page explains informational nature of the guidance, no guarantee of diagnosis, user responsibility for local label compliance, and emergency/agricultural expert escalation.
- Do not claim the app collects no data if images are transmitted to Gemini. Be honest about network processing.

7. Gemini API Integration Requirements

Use the Gemini API for image-and-text analysis. Implement a typed service layer; do not scatter API calls through UI components. Choose a currently supported vision-capable Gemini model available to the project and verify its current model ID, endpoint, request format, quotas, and terms at implementation time. Do not hardcode assumptions about free-tier availability.

7.1 Request and response contract

- Input: resized image, optional crop selected by user, optional symptoms/context, selected language, and strict instruction to return cautious agricultural guidance.
- Request structured JSON output where supported. Validate every response against a schema before rendering.
- Suggested fields: crop_guess, issue_title, assessment_level, visible_symptoms[], alternative_causes[], immediate_steps[], prevention_steps[], follow_up_questions[], escalation_advice, safety_notes[], limitations, language.
- Treat all model output as untrusted. Escape/render text safely; never execute model-provided code, URLs, HTML, or commands.
- If response JSON is invalid, attempt one bounded repair/parse strategy; then show a recoverable error. Do not silently invent missing fields.
- Use a short timeout and bounded retry with exponential backoff for transient errors. Do not retry quota errors repeatedly.
- Keep API usage visible in internal diagnostics without exposing secrets. Add request size limits and image compression.

7.2 Prompt safety and agricultural reliability

- Tell Gemini to identify only visible evidence, state uncertainty, list plausible look-alikes, and ask for more context when needed.

- Instruct it not to claim certainty from a single image, not to invent sources, and not to provide unsupported pesticide dosage/brand instructions.
- For high-risk or unclear cases, recommend local agricultural extension/officer support and additional photos.
- Use curated, locally relevant reference content to ground advice where possible. Do not assume the model always knows current Bangladesh product registrations.
- Add a user-facing disclaimer and never represent an AI-generated diagnosis as a lab-confirmed result.

7.3 API key handling

Never commit secrets to source control, logs, screenshots, generated artifacts, or the APK. Never expose a production API key in a client bundle. For a local-only prototype, use a local environment configuration and document that a client-side key is extractable. Before any public distribution, either deploy a minimal server-side proxy with verified no-cost limits and secret storage, or do not enable public API calls. Do not claim the app is secure merely because a key is obfuscated.

7.4 Graceful free-tier behavior

- When quota is exceeded, show a clear message and offer retry later, offline guides, or saving the image for later.
- Do not repeatedly call the API in a loop or automatically analyze images in the background.
- Allow a user to cancel an analysis. Avoid sending duplicate requests from repeated taps.
- Do not promise unlimited daily diagnoses. Record usage locally only for helpful feedback, not as a security boundary.

8. Zero-Budget Technical Architecture

Choose the simplest maintainable stack supported by Google AI Studio. If the build target is native Android, prefer Kotlin/Jetpack Compose and Room/DataStore where supported. If AI Studio produces a web-first app, use a modern responsive frontend and clearly state the exact path to Android packaging; do not pretend a web page is a native app. Keep the product architecture modular so UI can be retained if the stack changes.

- Presentation: design tokens, reusable app bar, bottom navigation, cards, buttons, dialogs, skeletons, error states, empty states.
- Domain: diagnosis use case, crop management, treatment checklist, reminders, history, localization.
- Data: local database/preferences, bundled curated guide content, image file storage with cleanup policy.
- AI service: Gemini client wrapper, request builder, schema validator, timeout/retry, quota/error mapper.
- Network: no required backend for local features. Any server proxy is optional and only after no-cost feasibility is verified.
- Observability: development-only error logging with personal data and API keys redacted.

9. Data Model and Persistence

Use stable IDs, timestamps, versioned local schema, migrations, and repository abstractions. No reset-on-launch behavior. Do not rely on in-memory arrays for user data.

9.1 Suggested entities

- PlantRecord: id, name, cropType, variety?, plantedAt?, fieldLabel?, notes?, photoPath?, createdAt, updatedAt, archivedAt?.
- DiagnosisRecord: id, createdAt, cropRecordId?, imagePath?, cropGuess?, issueTitle, assessmentLevel, symptoms[], alternativeCauses[], immediateSteps[], preventionSteps[], escalationAdvice?, rawStructuredResult?, language, modelId?, status.
- TreatmentStep: id, diagnosisId, title, description, orderIndex, isCompleted, completedAt?.
- CropTask: id, title, dueAt, cropRecordId?, notes?, repeatRule?, notificationEnabled, completedAt?, createdAt, updatedAt.

- AppPreferences: language, theme, onboardingCompleted, notificationPreferences, privacy/consent version, image retention preference.
- GuideArticle: id, crop, title, symptoms, lookAlikes, management, prevention, sources, reviewedAt, contentVersion.

9.2 Data behavior

- Every create/edit/delete operation updates persistent storage and the UI immediately.
- On relaunch, restore crops, history, tasks, language, theme, and onboarding state.
- Handle database migration without deleting existing records. Back up or preserve data when schema changes.
- Image storage must be size-limited and cleaned when related records are deleted, unless the user chooses to keep the image.
- Use explicit confirmation for irreversible bulk deletion.

10. Localization and Accessibility

- Default locale: English. Bangla (bn-BD) is the first additional locale.
- No hardcoded user-facing strings inside UI components. Use resource keys for all text, validation, error messages, accessibility labels, and notifications.
- Language selection must persist across restart and apply immediately without app restart where practical.
- Support Bangla text expansion and line wrapping; never clip headers or buttons. Allow two-line titles where needed.
- Format dates, times, numbers, and units according to the selected locale. Avoid concatenating sentence fragments that break grammar in translation.
- Use accessible contrast, screen-reader labels, scalable text, semantic controls, and non-color-only status indicators.
- Do not make a language toggle decorative: it must change the whole app.

11. Privacy, Consent, and Safety

- No mandatory account or login for MVP.
- Before the first AI upload, disclose that the selected image and provided context will be sent to Gemini for processing.
- Do not upload location, EXIF metadata, crop history, or unrelated images unless required and explicitly explained.
- Offer local deletion controls and explain whether images are retained on-device.
- Do not guarantee disease identification. Avoid unsafe chemical advice. Prefer low-risk cultural controls and expert referral for uncertain/high-impact cases.
- Do not claim professional agronomist endorsement unless such endorsement actually exists.
- Use source attribution for curated guidance and record review dates when available.

12. Error, Empty, and Edge States

- No crops yet → inviting empty state and Add Crop action.
- No history yet → explain that completed diagnoses will appear here.
- No internet → offline library and local features remain usable; analysis action explains the connection requirement.
- API quota exceeded → retry later and use offline guide.
- Uncertain diagnosis → ask for another photo/context and offer expert referral.
- Permission denied → explain why it is needed and offer a settings path or alternative.
- Storage full / image too large → show actionable guidance and avoid crashing.
- No notifications permission → reminders remain visible in app.
- Unsupported crop or poor image → do not force a disease label; request a clearer photo or more context.
- App update/schema migration failure → preserve existing data and show recoverable error where possible.

13. Implementation Phases

Phase 0 — Project audit and foundation

Inspect generated project, identify framework, set up design tokens, navigation, app shell, safe-area rules, persistence strategy, localization structure, and shared components. No fake features.

Phase 1 — Visual system and core navigation

Implement splash, onboarding, Home, bottom navigation, standard headers, loading/empty/error states, and responsive layout matching the master mockup.

Phase 2 — Local data features

Implement My Crops, crop library starter data, history models, profile/settings, local persistence, language switch, privacy/about/terms.

Phase 3 — Diagnosis workflow

Implement camera/gallery, image preview/compression, consent, Gemini service, structured result validation, uncertainty UI, retry/quota/error states.

Phase 4 — Treatment and follow-up

Implement treatment plans, step completion, follow-up Q&A, source-aware guide details, safety rules.

Phase 5 — Calendar and notifications

Implement user-created tasks/reminders and real local notifications with permissions and lifecycle handling.

Phase 6 — Polish and QA

Test all routes, persistence after restart, English/Bangla layout, header alignment, accessibility, performance, network failure, API quota failure, and release build.

14. QA and Acceptance Checklist

- App launches without crash; splash routes correctly; onboarding appears only when appropriate.
- All visible buttons and navigation items work; no dead UI, broken route, or accidental app exit.
- Home greeting reflects local time rather than theme.
- Header and system insets meet every requirement in Section 4 across multiple screen sizes.
- Camera capture and gallery selection work on a real device or are accurately labelled unsupported in the current prototype.
- Gemini analysis makes a real request when configured; failures are handled honestly; no fake success.
- API key is not committed or exposed in a public release build.
- Result content is schema-validated and safety notes appear where appropriate.
- Create/edit/delete crop, task, diagnosis, and treatment step; force-close and reopen; verify persistence.
- Switch English ↔ Bangla; verify all screens, dialogs, empty states, and long headers.
- Test airplane mode, slow network, denied permissions, quota limit, invalid API key, malformed model response, and large image.
- Privacy, Terms, About, and Help pages load and contain real content.
- No fabricated weather, fake health scores, fake diagnosis history, or mock notifications appear in production mode.
- No text or controls overflow, overlap, clip, or fall behind the bottom navigation/gesture area.
- Run a final visual pass against the master mockup and fix alignment/spacing before adding more features.

15. Final Build Rules

- Work phase by phase. First establish a clean, accurate, responsive visual foundation; then wire real behavior.
- Before implementing a feature, inspect whether it already exists. Preserve working code and data.
- Do not claim a feature is complete until it has been exercised end-to-end.
- Use realistic sample content only for design previews; clearly distinguish sample content from real user data and real AI output.
- Prioritize correctness, trust, data persistence, and smoothness over the number of features.
- Deliver a short implementation summary: what works, what remains, how to configure Gemini safely, and known limitations.

Appendix A — Example Gemini System Instruction

You are AgriCare, a cautious plant-health assistant. Analyze only the visible evidence in the provided plant/crop image and user context. Do not claim certainty from one image. If evidence is weak, say so and ask for a clearer photo or targeted context. Provide a likely issue, visible symptoms, plausible alternative causes, low-risk immediate actions, prevention, and escalation advice. Do not invent sources, registrations, product names, pesticide doses, or local legal approvals. Never give unsafe chemical mixing instructions. Distinguish general educational guidance from verified local label guidance. Return only schema-conforming JSON in the requested language. If the crop or condition is not identifiable, say unknown rather than guessing.

Appendix B — Required Structured Result Schema (conceptual)

{"crop_guess":"string or unknown","issue_title":"string","assessment_level":"likely | uncertain | insufficient_image","visible_symptoms":["string"],"alternative_causes":["string"],"immediate_steps":[{"title":"string","description":"string","priority":"now | soon | monitor"}],"prevention_steps":["string"],"follow_up_questions":["string"],"escalation_advice":"string","safety_notes":["string"],"limitations":"string","language":"en | bn-BD"}

Appendix C — Reference Image Rule

The latest user-approved AgriCare UI mockup collage in this document is the master visual reference. Use it as the default visual direction for splash, onboarding, Home, camera, analysis result, treatment, crop library, history, calendar, notifications, voice assistant, and profile. Individual screens may be refined for actual platform constraints, but do not change the overall visual identity without explicit user approval.